

RACTF 2020 Crypto - B007L36 CRYPT70 A641N

deuterium — 2020-06-09

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/ractf/crypto/B007L36-CRYPT70-A641N/2020-06-09-RCTF-2020-B007L36-CRYPT70-4641N.html>

Abstract

CTF writeup: RACTF 2020's B007L36 CRYPT70 A641N puts on Base64 camouflage. Underneath, repeating-key addition lets known plaintext recover the key by subtraction.

B007L36 CRYPT70... 4641N

After finally solving B007L3G CRYPT70, we are again presented with another variant. The description reads as follows:-

As you continue your pentest of MEGACORP, you make your way to an admin-only subnet of the network. There, you find yet another custom crypto implementation. You also previously found this zip file on a user's desktop. Solving this may be the last step to gaining full access to the company's network

And encryption.service.zip, the contents of which are :-

ciphertext.txt

```
w4bDKMKDw6jDi80uw6JQw6jDh80ZwojCmM0Nw4nDnsKtwqnDk80iwqLDosKdw6XDhs0Vw6
rDj80ew5NcwpTDhM0iw4vCpc0Yw5bDoFTCrC0Hw6LCpsKUw6PDM80Nw4jCLM0dw6TDosKY
wpTDM0jw53CpX/Dic0bwqHCqc0Aw6fCrMKUw6bDpc0Uw5jDmc0KwvpDocKVw5fDkc0Zw5
xTw4rDi80lVMKaw43DnVPDmc0rw6XDls0Vw5nChs0vw5bCkc0of80dw5xTw5HDi80fwqnC
pc0Tw6xTw53Dq8KSw5XDi80ZwobDns0XwqDDnM0Ew6bDnMKYw5fDmsKawqjCsc0TwpnCmc
0dw6nDl8KP
```

plaintext.txt

To test the encryption service, encrypt this file with your company issued secret key and ensure that it results in the ciphertext.txt file.

And a password.txt

```
w6TDgs0Gw6jDjM02w5RgwqTDi80Tw5Vmr7Cnc0jZcKcwpLDmGjDnMKxw5/CLM0CwqTDLM0aw5tjw7E=
```

Lets first begin with by checking some stuff like we did in B007L3G CRYPT70 with the encryption service

Please enter the secret key to encrypt the data with: a
Please enter the data that you would like to encrypt: a
Your encrypted message is: w4I=

```
Please enter the secret key to encrypt the data with: b
Please enter the data that you would like to encrypt: a
Your encrypted message is: w4M=
```

Notice the encrypted message appears to be base64 encoded.

Lets check the decoded values

```
from base64 import b64decode as d64
d64('w4M=')
# b'\xc3\x83'
d64('w4I=')
# b'\xc3\x82'
```

EWV, looks odd probably because of unicode. Lets deal with string-objects instead of byte-objects by simply calling `decode()`

```
from base64 import b64decode as d64
d64('w4M=').decode()
# 'Ã'
d64('w4I=').decode()
# 'Â'
ord('Ã')
# 195
ord('Â')
# 194
```

Hehe, we start getting a hint already, changing the key by 1, encryption changes by 1. There is probably something *Very Linear* about the encryption.

To help with encryption process, I created a small helper function `encrypt` which calls `encrypt` on the server and return the ord values of base64 decoded string.

```
from pwn import remote
from base64 import b64decode as d64
from base64 import b64encode as e64

HOST, PORT = "95.216.233.106", 60246

with open('password.txt', 'r') as password_file:
    password = d64(password_file.read().strip()).decode()

with open('ciphertext.txt', 'r') as ciphertext_file:
    ct = d64(ciphertext_file.read().strip()).decode()

with open('plaintext.txt', 'r') as plaintext_file:
    pt = plaintext_file.read().strip()

REM = remote(HOST, PORT)
print(REM.recvline())
print(REM.recvline())

def encrypt(pt, key):
    REM.recvuntil(b'data with:')
    REM.sendline(key)
```

```
REM.sendline(pt)
data = REM.recvuntil(b'\n\n')
encrypted = data.split(b'message is: ')[-1].strip()
return [ord(i) for i in d64(encrypted).decode()]
```

Lets play around a little bit more...

```
>>> encrypt(b'aaaa', 'a')
[194, 194, 194, 194]
>>> encrypt(b'aaaa', 'ab')
[194, 195, 194, 195]
>>> encrypt(b'aaaa', 'abcd')
[194, 195, 196, 197]
>>> encrypt(b'abcd', 'aaaa')
[194, 195, 196, 197]
```

We can make the following observations :-

- key is repeated if plaintext is longer than the key
- The encryption is changed only by 1 on varying either key or plaintext by 1
- The key, plaintext and ciphertext are related just by an addition modulo 256

So, all we need to do now is to figure out what the provided files mean.

Let us try checking the key with which ciphertext.txt and plaintext.txt are related.

```
ct_arr = [ord(i) for i in ct]
pt_arr = [ord(i) for i in pt]
diff_arr = [ct_arr[i] - pt_arr[i] for i in range(len(ct_arr))]
print("".join(chr(i) for i in diff_arr))
```

ractf{n0t_th3_fl49_y3t}ractf{n0t_th3_fl49_y3t}ractf{n0t_th3_fl49_y3t}ractf{n0t_th3_fl49_y3t}ractf{n0t_th3_fl49_y3t}ractf{n0t_th3_fl49_y3t}ra

YES, it indeed verifies all our hypothesis and we now know the company issued secret key is ractf{n0t_th3_fl49_y3t}

Lets quickly decrypt the contents of password.txt with the key

```
key = "ractf{n0t_th3_fl49_y3t}"
flag = "".join( chr(ord(password[i]) - ord(key[i%len(key)])) for i in
range(len(password)) )
print(flag)
```

And we finally get our flag ractf{f00l_m3_7wlc3_5h4m3_0n_m3}

CHEERS YOU ALL!

Interactive controls are omitted from this PDF. See the canonical HTML version.